

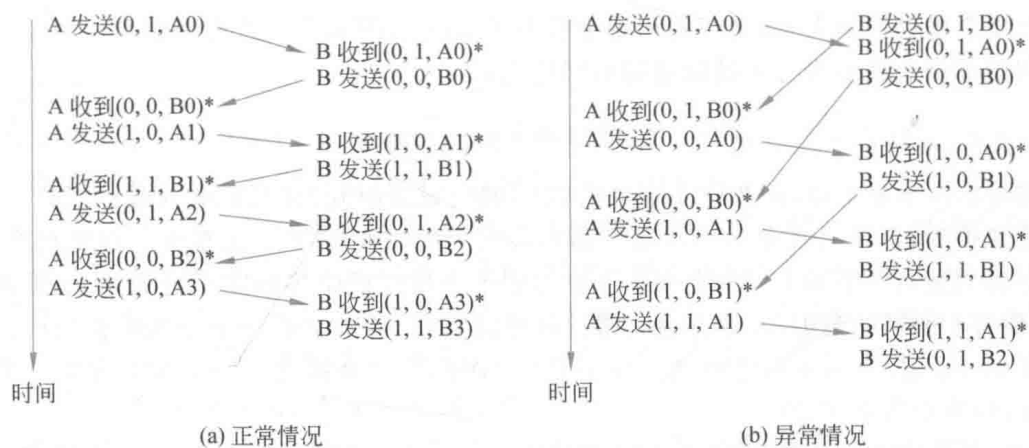


图 3-17 协议 4 的正常情形和异常情形

三元组表示(序号, 确认, 包号), 星号表示网络层收到了一个包

回退 n 协议

到现在为止, 本书一直有这样的假设, 即一个帧到达接收方需要的传输时间加上确认回来的传输时间可以忽略不计。有时候, 这种假设明显是不对的。在这些情形下, 过长的往返时间对于带宽的利用效率有重要的影响。举一个例子, 考虑一个 50kb/s 的卫星信道, 它的往返传播延迟为 500ms。想象一下, 通过该卫星用协议 4 发送长度为 1000 位的帧。在 $t=0$ 时, 发送方开始发送第一帧; 在 $t=20\text{ms}$ 时, 该帧被完全发送出去; 直到 $t=270\text{ms}$ 时该帧才完全到达接收方; 在 $t=520\text{ms}$ 时, 确认帧才到达发送方, 这都是在最好情况下的时间 (即在接收方没有停顿并且确认帧很短)。这意味着, 发送方在 $500/520$ 或 96% 的时间内是被阻塞的。换句话说, 只有 4% 的有效带宽被利用了。很显然, 从效率的角度看, 长发送时间、高带宽和短帧这三者组合在一起是一种灾难。

这里描述的问题可以看作这种规则的必然结果, 即发送方在发送下一帧之前必须等待前一帧的确认。如果放宽这一限制, 则可以获得更高的效率。基本的思想是: 这个方案允许发送方在阻塞之前发送多达 w 个帧, 而不只是一个帧。通过选择足够大的 w 值, 发送方就可以连续发送帧, 因为在发送窗口被填满之前, 前面的帧的确认就到来了, 从而防止了发送方被阻塞。

为了找到一个合适的 w 值, 需要知道在一帧从发送方传送到接收方期间信道上能容纳多少个帧。这个容量是由带宽乘以单向传送时间决定的, 或称为链路的带宽-延迟乘积 (bandwidth-delay product)。可以将这个数量除以一帧的位数, 从而将它表示成帧的数量, 将这个数值称为 BD。然后, w 应设置为 $2BD+1$ 。如果考虑发送方连续发送帧并且在往返时间内收到一个确认, 那么 $2BD$ 就是发送方可以连续发出去的帧的个数, 加 1 是因为必须接收完整个帧之后确认帧才会被发出。

在上述例子中, 对于具有 50kb/s 带宽和 250ms 单向传输时间的链路, 带宽-延迟乘积为 12.5kb 或 12.5 个长度为 1000 位的帧。于是, $2BD+1$ 等于 26。假设发送方还和以前一样开始发送 0 号帧, 并且每隔 20ms 发送一个新帧。到 $t=520\text{ms}$ 时, 它已经发送了 26 帧, 这时 0 号帧的确认刚好到来。此后, 每隔 20ms 就会到达一个确认, 因此发送方在需要时总是能够继续发送帧。从那时起, 25 个或 26 个未被确认的帧将始终在传输途中。换言之, 发送方的最大窗口大小是 26。

对于更小的窗口大小,链路的利用率将小于 100%,因为发送方有时会被阻塞。可以将链路利用率表示成发送方未被阻塞的时间比例:

$$\text{链路利用率} \leq \frac{w}{1+2BD}$$

该公式求出的值是上限,因为它不容许有任何的帧处理时间,并且视确认帧的长度为 0(因为它通常很短)。该公式显示当带宽-延迟乘积大的时候窗口 w 也要大。如果延迟很高,发送方将迅速耗尽其窗口,即使对于卫星的例子中那样的中等带宽也是如此;如果带宽很大,即使对于普通的延迟,发送方也将很快耗尽其窗口,除非它有一个很大的窗口(比如,一个具有 1ms 延迟的 1Gb/s 链路能容纳 1Mb)。对于停-等式协议, $w=1$,即使延迟只有一帧的时间,效率也将低于 50%。

保持多个帧同时在传输途中的技术是管道化(pipelining)的一个例子。在一条不可靠的通信信道上像管道一样传送帧会引起一些严重的问题。如果在一个长的数据流的中间某一个帧被损坏或丢失,会发生什么事情?在发送方发现错误以前,大量的后续帧将到达接收方。当损坏的那个帧到达接收方时,显然它应该被丢弃,但接收方该如何处理那些后续到达的正确帧呢?请记住,接收方的数据链路层有责任按正确的顺序把数据包传递给网络层。

有两种基本办法可用来处理管道化传输中出现的错误,图 3-18 显示了这两种技术。

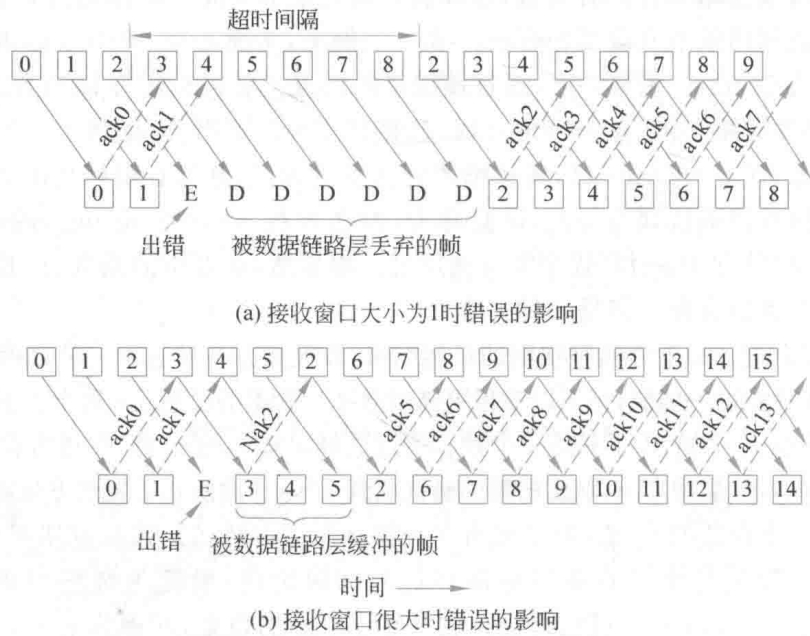


图 3-18 管道化以及错误恢复

一种选择称为回退 n (go-back- n)协议,接收方只需简单丢弃所有后续的帧,而且对这些丢弃的帧不发送确认。这种策略对应于接收窗口大小为 1。换句话说,除了数据链路层必须要递交给网络层的下一帧以外,它拒绝接收任何帧。如果在计时器超时以前,发送方的窗口已被填满,则管道将开始空闲。最终,发送方将超时,并且按照顺序重传所有未被确认的帧,从那个受损或者被丢失的帧开始。如果错误率很高,那么这种方法将会浪费大量的带宽。

在图 3-18(a)中,可以看到回退 n 帧的情形,其中接收方的窗口为 1。0 号帧和 1 号帧被正确地接收和确认。然而,2 号帧被损坏或者丢失。发送方并不知道出现了问题,它继续发送帧,直到 2 号帧的计时器超时。然后,它回退到 2 号帧,从它开始重新发送 2 号帧、3 号

帧、4号帧……一切从头再来。

选择重传

如果错误很少发生,则回退 n 协议可以工作得很好;但是如果线路质量差,它就会浪费很多带宽用于重传帧。我们需要做得比这更好,这是有可能的。另一种策略是选择重传(selective repeat),即允许接收方接受和缓冲那些在损坏帧或丢失帧之后正确接收到的帧。

当使用这种策略时,接收到的坏帧被丢弃,但之后接收到的任何好帧都被接受并缓冲起来。当发送方超时,只有最早的未被确认的那个帧被重传。如果该帧正确到达,则接收方就可按顺序将它缓冲的所有帧递交给网络层。选择重传对应的接收方窗口大于1。如果窗口很大,则这种方法要求大量的数据链路层内存。

选择重传通常跟这样的策略结合起来:当接收方检测到错误(比如帧的校验和错误或者序号不正确)时,它发送一个否定确认(Negative Acknowledgement, NAK)。NAK 可以触发该帧的重传操作,而不需要等到相应的计时器超时,因此可以提高协议性能。

在图 3-18(b)中,0号帧和1号帧被正确接收,并得到确认;2号帧丢失了。当3号帧到达接收方时,那里的数据链路层注意到自己错过了一帧,所以它针对2号帧返回一个NAK,但是将3号帧缓冲起来。当4号和5号帧到达时,它们也被数据链路层缓冲起来,而没有传递给网络层。最终,2号帧的NAK抵达发送方,它立即重新发送2号帧。当该帧到达时,数据链路层现在有了2号帧、3号帧、4号帧和5号帧,于是就将它们按照正确的顺序传递给网络层。它也可以确认所有这些帧,包括5号帧,如图 3-18(b)所示。如果NAK被丢失,则发送方的2号帧计时器最终超时,发送方就会重新发送2号帧(仅仅这一帧),但是,这可能已经过了相当长一段时间了。

这两种方法恰好反映了带宽使用效率和数据链路层缓冲区空间之间的权衡。根据哪个资源更为紧缺,选择使用其中一种方法。图 3-19 显示了一个回退 n 协议——协议 5,其中接收方的数据链路层按顺序接收到达的帧,发生错误之后的帧都被丢弃。在这个协议中,第一次抛掉了网络层总是有无穷多的数据包要发送的假设。当网络层有数据包要发送时,它可以触发一个 network_layer_ready 事件。为了强制对发送窗口进行流量限制,或者对任何时候尚在途中未被确认的帧的数量进行限制,数据链路层必须能够阻止网络层给予它过多的工作。库过程 enable_network_layer() 和 disable_network_layer() 可以完成这样的任务。

```
/* Protocol 5 (Go-back-n) allows multiple outstanding frames. The sender may
transmit up to MAX_SEQ frames without waiting for an ack. In addition,
unlike in the previous protocols, the network layer is not assumed to have
a new packet all the time. Instead, the network layer causes a network_layer
_ready event when there is a packet to send. */

#define MAX_SEQ 7
typedef enum {frame_arrival, cksum_err, timeout, network_layer_ready} event_type;
#include "protocol.h"
static boolean between(seq_nr a, seq_nr b, seq_nr c)
{
    /* Return true if a <= b < c circularly; false otherwise. */
    if (((a <= b) && (b < c)) || ((c < a) && (a <= b)) || ((b < c) && (c < a)))
        return(true);
    else
        return(false);
}
static void send_data(seq_nr frame_nr, seq_nr frame_expected, packet buffer[ ])
```

图 3-19 一个回退 n 协议——协议 5

```

{
/* Construct and send a data frame. */
frame s; /* scratch variable */
s.info = buffer[frame_nr]; /* insert packet into frame */
s.seq = frame_nr; /* insert sequence number into frame */
s.ack = (frame_expected + MAX_SEQ) % (MAX_SEQ + 1); /* piggyback ack */
to_physical_layer(&s); /* transmit the frame */
start_timer(frame_nr); /* start the timer running */
}
void protocol5(void)
{
seq_nr next_frame_to_send; /* MAX_SEQ > 1; used for outbound stream */
seq_nr ack_expected; /* oldest frame as yet unacknowledged */
seq_nr frame_expected; /* next frame expected on inbound stream */
frame r; /* scratch variable */
packet buffer[MAX_SEQ + 1]; /* buffers for the outbound stream */
seq_nr nbuffered; /* number of output buffers currently in use */
seq_nr i; /* used to index into the buffer array */
event_type event;
enable_network_layer(); /* allow network_layer_ready events */
ack_expected = 0; /* next ack expected inbound */
next_frame_to_send = 0; /* next frame going out */
frame_expected = 0; /* number of frame expected inbound */
nbuffered = 0; /* initially no packets are buffered */
while (true) {
wait_for_event(&event); /* four possibilities: see event_type above */
switch(event) {
case network_layer_ready: /* the network layer has a packet to send */
/* Accept, save, and transmit a new frame. */
from_network_layer(&buffer[next_frame_to_send]); /* fetch new packet */
nbuffered = nbuffered + 1; /* expand the sender's window */
send_data(next_frame_to_send, frame_expected, buffer); /* transmit the
frame */ inc(next_frame_to_send); /* advance sender's upper window edge */
break;
case frame_arrival: /* a data or control frame has arrived */
from_physical_layer(&r); /* get incoming frame from physical layer */
if (r.seq == frame_expected) {
/* Frames are accepted only in order. */
to_network_layer(&r.info); /* pass packet to network layer */
inc(frame_expected); /* advance lower edge of receiver's window */
}
/* Ack n implies n - 1, n - 2, etc. Check for this. */
while (between(ack_expected, r.ack, next_frame_to_send)) {
/* Handle piggybacked ack. */
nbuffered = nbuffered - 1; /* one frame fewer buffered */
stop_timer(ack_expected); /* frame arrived intact; stop timer */
inc(ack_expected); /* contract sender's window */
}
break;
case cksum_err: break; /* just ignore bad frames */
case timeout: /* trouble; retransmit all outstanding frames */
next_frame_to_send = ack_expected; /* start retransmitting here */
for (i = 1; i <= nbuffered; i++) {
send_data(next_frame_to_send, frame_expected, buffer); /* resend frame */
inc(next_frame_to_send); /* prepare to send the next one */
}
}
if (nbuffered < MAX_SEQ)
enable_network_layer();
else
disable_network_layer();
}
}

```

图 3-19 (续)

任何时候尚在传输途中的帧的最大数量不能与序号空间的大小相同。对回退 n 协议,任何时候都可以有 MAX_SEQ 个帧还在传输途中未被确认,即使存在 $\text{MAX_SEQ}+1$ 个不同的序号(分别为 $0, 1, 2, \dots, \text{MAX_SEQ}$)。接下来就会看到,对于下一个选择重传协议,这种限制更为严格。为了看清楚为什么要求有这个限制,请考虑下面 $\text{MAX_SEQ}=7$ 的情形:

- (1) 发送方发送 $0 \sim 7$ 号帧。
- (2) 7 号帧的捎带确认返回发送方。
- (3) 发送方发送另外 8 个帧,其序号仍然是 $0 \sim 7$ 。
- (4) 现在 7 号帧的另一个捎带确认也进来了。

问题出现了:第二批的 8 个帧是全部成功到达了还是全部丢失了(把出错之后丢弃的帧也算作丢失)?在这两种情况下,接收方都会发送针对 7 号帧的确认。但发送方却无法分辨。由于这个原因,未确认帧的最大数目必须限制为不能超过 MAX_SEQ (不是 $\text{MAX_SEQ}+1$)。

虽然协议 5 没有缓冲出错后到来的帧,但是它也没有完全摆脱缓冲问题。由于发送方可能在将来的某个时刻要重传所有未被确认的帧,所以,它必须把已经发送出去的帧一直保留,直到它能确定接收方已经接受了这些帧。

当 n 号帧的确认到达时, $n-1$ 号帧、 $n-2$ 号帧等都会自动被确认。这种类型的确认称为累计确认(cumulative acknowledgement)。当先前一些捎带确认的帧被丢失或者受损以后,这个特性显得尤为重要。每当任何一个确认到达,数据链路层都要检查是否可以释放一些缓冲区。如果能释放一些缓冲区(即窗口中又有了一些可以利用的空间),则原来被阻塞的网络层又允许触发更多的 `network_layer_ready` 事件了。

对于这个协议,假设链路上总是有反向的流量可以捎带确认。协议 4 不需要这样的假设,因为它每接收到一帧就送回一帧,即使它刚刚已经发送出去一帧。在下一个协议中,我们将用一种非常巧妙的办法解决这一单向流量问题。

因为协议 5 有多个未被确认的帧,所以逻辑上它需要多个计时器,即每一个未被确认的帧都需要一个计时器。每一帧的超时是独立的,相互之间没有关系。然而,用软件很容易模拟所有这些计时器:只需使用一个硬件时钟,它周期性地引发中断。所有未发生的超时构成了一个链表,链表中的每个节点包含了离超时到期的剩余时钟滴答数、超时对应的帧以及一个指向下一个节点的指针。

为了示范如何实现多个计时器,请考虑图 3-20(a)中的例子。假设每 1ms 时钟滴答一次。初始时,实际时间为 $10:00:00.000$;有 3 个超时事件正在进行中,分别定在 $10:00:00.005$ 、 $10:00:00.013$ 和 $10:00:00.019$ 。每当时钟滴答一次,实际的时间就被更新,链表头上的剩余时钟滴答数减 1。当滴答数变成 0 时,就引发一个超时事件,并将该节点从链表中移除,如图 3-20(b)所示。虽然这种组织方式要求在 `start_timer()` 或 `stop_timer()` 被调用时扫描链表,但在每次滴答时并不要求更多的工作。在协议 5 中, `start_timer()` 和 `stop_timer()` 这两个例程都带一个参数,指明针对哪一帧计时。

在这个协议中,发送方和接收方分别维护一个已发出但未确认的序号和可接受序号的窗口。发送方的窗口大小从 0 开始,以后可以增大到某一个预设的最大值。相反,接收方的窗口总是固定不变的,其大小等于预先设定的最大值。接收方为其窗口内的每个序号保留一个缓冲区。与每个缓冲区相关联的还有一个标志位(arrived),指明了该缓冲区是满的还



图 3-20 软件模拟多个计时器

是空的。每当一帧到来,接收方通过 `between` 函数检查它的序号,看是否落在窗口内。如果确实落在窗口内,并且以前没有接收过该帧,则接受该帧,并且保存起来。不管该帧是否包含了网络层期望的下一个数据包,这个动作都要执行。当然,该帧必须被保存在数据链路层内部,直到所有序号比它小的那些帧都已经按顺序递交给网络层以后,它才能被传递给网络层。图 3-21 给出了一个使用该算法的协议——协议 6。

```

/*Protocol 6 (Selective repeat) accepts frames out of order but passes packets to
the network layer in order. Associated with each outstanding frame is a timer.
When the timer expires, only that frame is retransmitted, not all the outstanding
frames, as in protocol 5. */

#define MAX_SEQ7 /* should be 2^n - 1 */
#define NR_BUFS ((MAX_SEQ + 1)/2)
typedef enum {frame_arrival, cksum_err, timeout, network_layer_ready, ack_timeout}
event_type;
#include "protocol.h"
boolean no_nak = true; /* no nak has been sent yet */
seq_nr oldest_frame = MAX_SEQ + 1; /* initial value is only for the simulator */
static boolean between(seq_nr a, seq_nr b, seq_nr c)
{
    /* Same as between in protocol 5, but shorter and more obscure. */
    return ((a <= b) && (b < c)) || ((c < a) && (a <= b)) || ((b < c) && (c < a));
}
static void send_frame(frame_kind fk, seq_nr frame_nr, seq_nr frame_expected,
packet buffer[])
{
    /* Construct and send a data, ack, or nak frame. */
    frame s; /* scratch variable */
    s.kind = fk; /* kind == data, ack, or nak */
    if (fk == data) s.info = buffer[frame_nr % NR_BUFS];
    s.seq = frame_nr; /* only meaningful for data frames */
    s.ack = (frame_expected + MAX_SEQ) % (MAX_SEQ + 1);
    if (fk == nak) no_nak = false; /* one nak per frame, please */
    to_physical_layer(&s); /* transmit the frame */
    if (fk == data) start_timer(frame_nr % NR_BUFS);
    stop_ack_timer(); /* no need for separate ack frame */
}
void protocol6(void)
{
    seq_nr ack_expected; /* lower edge of sender's window */
    seq_nr next_frame_to_send; /* upper edge of sender's window + 1 */
    seq_nr frame_expected; /* lower edge of receiver's window */
    seq_nr too_far; /* upper edge of receiver's window + 1 */
    int i; /* index into buffer pool */
    frame r; /* scratch variable */

```

图 3-21 一个选择重传协议——协议 6

```

packet out_buf[NR_BUFS];          /* buffers for the outbound stream */
packet in_buf[NR_BUFS];           /* buffers for the inbound stream */
boolean arrived[NR_BUFS];         /* inbound bit map */
seq_nr nbuffered;                 /* how many output buffers currently used */
event_type event;
enable_network_layer();           /* initialize */
ack_expected = 0;                 /* next ack expected on the inbound stream */
next_frame_to_send = 0;           /* number of next outgoing frame */
frame_expected = 0;
too_far = NR_BUFS;
nbuffered = 0;                   /* initially no packets are buffered */
for (i = 0; i < NR_BUFS; i++) arrived[i] = false;
while (true) {
    wait_for_event(&event);        /* five possibilities: see event_type above */
    switch(event) {
        case network_layer_ready: /* accept, save, and transmit a new frame */
            nbuffered = nbuffered + 1; /* expand the window */
            from_network_layer(&out_buf[next_frame_to_send % NR_BUFS]);
            /* fetch new packet */
            send_frame(data, next_frame_to_send, frame_expected, out_buf);
            /* transmit the frame */
            inc(next_frame_to_send); /* advance upper window edge */
            break;
        case frame_arrival:       /* a data or control frame has arrived */
            from_physical_layer(&r); /* fetch incoming frame from physical layer */
            if (r.kind == data) {
                /* An undamaged frame has arrived. */
                if ((r.seq != frame_expected) && no_nak)
                    send_frame(nak, 0, frame_expected, out_buf); else start_ack_timer();
                if (between(frame_expected, r.seq, too_far) && (arrived[r.seq % NR_BUFS] ==
                    false)) {
                    /* Frames may be accepted in any order. */
                    arrived[r.seq % NR_BUFS] = true; /* mark buffer as full */
                    in_buf[r.seq % NR_BUFS] = r.info; /* insert data into buffer */
                    while (arrived[frame_expected % NR_BUFS]) {
                        /* Pass frames and advance window. */
                        to_network_layer(&in_buf[frame_expected % NR_BUFS]);
                        no_nak = true;
                        arrived[frame_expected % NR_BUFS] = false;
                        inc(frame_expected); /* advance lower edge of receiver's window */
                        inc(too_far); /* advance upper edge of receiver's window */
                        start_ack_timer(); /* to see if a separate ack is needed */
                    }
                }
            }
            if ((r.kind == nak) && between(ack_expected, (r.ack+1) % (MAX_SEQ+1), next_frame_to_send))
                send_frame(data, (r.ack+1) % (MAX_SEQ + 1), frame_expected, out_buf);
            while (between(ack_expected, r.ack, next_frame_to_send)) {
                nbuffered = nbuffered - 1; /* handle piggybacked ack */
                stop_timer(ack_expected % NR_BUFS); /* frame arrived intact */
                inc(ack_expected); /* advance lower edge of sender's window */
            }
            break;
        case cksum_err:
            if (no_nak) send_frame(nak, 0, frame_expected, out_buf); /* damaged frame */
            break;
        case timeout:
            send_frame(data, oldest_frame, frame_expected, out_buf); /* we timed out */
            break;
        case ack_timeout:
            send_frame(ack, 0, frame_expected, out_buf); /* ack timer expired; send ack */
    }
    if (nbuffered < NR_BUFS) enable_network_layer(); else disable_network_layer();
}
}

```

图 3-21 (续)

与只能按顺序接受帧的协议相比,非顺序接收的协议引入了有关帧序号的更多限制。下面用一个例子就很容易说明麻烦之处。假设用3位序号,那么发送方允许连续发送最多7个帧,然后开始等待确认。

刚开始时,发送方和接收方的窗口如图3-22(a)所示。现在发送方发出0~6号帧。接收方的窗口允许它接受任何序号为0~6的帧。这7个帧全部正确地到达了,所以接收方对它们进行确认,并且向前移动它的窗口,允许接收7、0、1、2、3、4或5号帧,如图3-22(b)所示。所有这7个缓冲区都标记为空。



图 3-22 发送和接收帧

此时,灾难降临了,就像闪电击中了电线杆子,所有的确认都被摧毁。协议应该不管灾难是否发生都能正确工作。最终发送方超时,并且重发0号帧。当0号帧到达接收方时,接收方进行检查,看它是否落在接收方的窗口中。不幸的是,如图3-22(b)所示,0号帧落在新窗口中,所以它被当作一个新帧接受了。接收方也返回(捎带)6号帧确认,因为0~6号帧都已经接收到了。

发送方得知所有它发出去的帧都已经正确地到达了,所以它向前移动窗口,并立即发送7、0、1、2、3、4和5号帧。7号帧将被接收方接受,并且它的数据包直接传递给网络层。紧接着,接收方的数据链路层进行检查,看它是否已经有一个有效的0号帧,它发现确实已经有了,然后它把缓冲区中原有的数据包作为新的数据包传递给网络层。因此,网络层得到了一个不正确的数据包。协议失败!

这个问题的本质在于:当接收方向前移动它的窗口后,新的有效序号范围与老的序号范围有重叠。因此,后续的一批帧可能是重复的帧(如果所有的确认都丢失了),也可能是新的帧(如果所有的确认都接收到了)。接收方根本无法区分这两种情形。

解决这个难题的办法是,确保接收方向前移动它的窗口以后,新窗口与原来的窗口没有重叠。为了保证没有重叠,窗口的大小应该最多是序号范围对应的序号数量的一半。这种情形如图3-22(c)和图3-22(d)所示。如果用3位表示序号,则序号范围为0~7。在任何时候,应该只有4个未被确认的帧。按照这种方法,如果接收方已经接收了0~3号帧,并且向前移动了窗口,以便允许接收第4~7帧,那么,它可以毫无二义地区分出后续的帧是重传帧(序号为0~3)还是新帧(序列号为4~7)。一般来说,协议6的窗口大小为 $(MAX_SEQ + 1)/2$ 。

一个有意思的问题是:接收方必须拥有多少个缓冲区?在任何情况下,接收方都不会接受序号低于窗口下界的帧,也不接受序号高于窗口上界的帧。因此,接收方需要的缓冲区的数量等于窗口的大小,而不是序号范围对应的序号数量。在前面的3位序号例子中,只需要4个缓冲区就够了,编号为0~3。当*i*号的帧到达时,它被放在*i mod 4*号缓冲区中。请

注意,虽然 i 和 $(i+4) \bmod 4$ 在“竞争”同一个缓冲区,但它们永远不会同时落在窗口内,因为那样就暗示着窗口的尺寸至少为 5。

出于同样的原因,需要的计时器数量等同于缓冲区的数量,而不是序号空间的大小。实际上,每个缓冲区都有一个与之关联的计时器。如果计时器超时,缓冲区的内容就要被重传。

协议 6 还放宽了隐含假设,即信道的负载很重。在协议 5 中作了这个假设,当时要依赖反向的帧捎带确认。如果反向流量很轻,确认可能会被延缓很长一段时间,这将导致问题。在极端情况下,如果在一个方向上有很大的流量,而另一个方向上没有流量,那么,当发送方的窗口达到最大值时协议将被阻塞。

为了放宽这一假设,当一个按正常顺序发送的数据帧到达以后,接收方通过 `start_ack_timer()` 启动一个辅助的计时器。如果在计时器超时以前没有出现反向流量,则发送一个单独的确认帧。由该辅助计时器超时而导致的中断称为 `ack_timeout` 事件。基于这样的处理方式,因为缺少可以捎带确认的反向数据帧不再是一个障碍,所以只存在单向数据的流量也是可能的。只要一个辅助计时器就可以正常工作,如果该计时器在运行时又调用了 `start_ack_timer()`,则该次调用没有影响。计时器不会被重置或者被扩展,因为它的作用是提供某种最小的确认率。

本质的一点是,辅助计时器的超时间隔应该明显小于用于数据帧计时器的超时值。为了确保一个被正确接收的帧尽早地得到确认,从而该帧的重传计时器不会超时,因而不会重传该帧,这一条件是必要的。

协议 6 采用了比协议 5 更加有效的策略来处理错误。任何时候当接收方有理由怀疑出现了错误时,它就给发送方送回一个否定确认(NAK)帧。这样的帧实际上是一个重传请求,在 NAK 中指定了要重传的帧。在两种情况下,接收方要特别留意:当一个受损的帧到来时,或者非期望的帧到来时(可能丢帧了)。为了避免多次请求重传同一个丢失帧,接收方应该记录对于某一帧是否已经发送过 NAK。在协议 6 中,如果还没有发送过 `frame_expected` 的 NAK,则变量 `no_nak` 为 `true`。如果 NAK 被损坏了或者丢失了,则不会有实际的损害,因为发送方最终会超时,无论如何它都会重传丢失的帧。如果一个 NAK 被发送出去并丢失以后错误的帧到来了,那么,`no_nak` 将为 `true`,并且辅助计时器将被启动。当该辅助计时器超时时,一个 ACK 帧将被发送出去,从而对发送方和接收方的当前状态进行同步。

在某些情形下,从一帧被发出去开始算起,到该帧经传播抵达目的地并在那里被处理,然后它的确认被传回来,整个过程需要的时间(几乎)是个常数。在这些情形下,发送方可以把它的计时器调得“紧”一些,让它恰好略微大于正常情况下从发送一帧到接收到其确认之间的时间间隔。在这种情况下 NAK 就没有用处了。

然而,在其他一些情况下这段往返时间高度可变。比如,在反向流量零零散散的情况下,如果刚好有反向流量,则确认之前的这段时间会比较短;反之,如果没有反向流量,则这段时间会很长。发送方面临着选择:要么将计时器的间隔设置得比较小(其风险是不必要的重传),要么将它设置得比较大(发生错误之后长时间地空等)。这两种选择都会浪费带宽。一般来说,如果确认间隔的标准偏差与间隔本身相比非常大,则计时器应该设置得“松”一点。然后,NAK 可以显著加快丢失帧或者损坏帧的重传速度。